Project Interim Report

On

File Compression Simulator
(Using Core Java & OOP Concepts)

RU-100-01-00012

Object Oriented Programming With Java

SESSION: 2025-26

Submitted By

ROHIT RAJ

ERP = 11205

Bachelor of Technology in Computer Science & Engineering with association with GOOGLE

Under the Guidance of

Mr. ASHISH SHIVHAARE

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(April 2026)

Abstract

The Movie Ticket Booking System is a console-based Java application designed to simulate the functionality of a real-world cinema seat reservation system. In modern cinema environments, efficient seat management is critical to prevent double bookings and ensure a smooth user experience. This project addresses that need by implementing a fully functional booking system using Object-Oriented Programming (OOP) principles in Core Java.

The system allows users to browse available movies, view a 2D seat layout, select seats by specifying row and column, and receive a booking confirmation along with the total cost. Seats are categorised as Regular or Premium with different pricing. The application strictly prevents double booking through status checks on each seat object. It serves as a practical demonstration of OOP concepts such as classes, object interaction, 2D arrays, encapsulation, and modular design — fulfilling the academic objectives of the Object Oriented Programming With Java course.

Introduction

Online ticket booking has become an essential part of the modern entertainment industry. Cinemas require robust systems that can manage seat availability in real time, handle concurrent bookings, and provide users with a seamless experience. This project replicates those features at a conceptual level using Core Java.

The Movie Ticket Booking System consists of three core classes — **Movie**, **Seat**, and **Booking** — each encapsulating specific responsibilities. Key features include:

- Display of available movies and show timings
- Visual 2D seat layout (O = Available, X = Booked)
- Seat selection by row and column index
- Prevention of double booking with real-time validation
- Automatic cost calculation based on seat type (Regular / Premium)
- Booking confirmation with complete details

Objectives

The primary objectives of this project are:

- Simulate a real-world ticket booking system in Java
- Apply OOP concepts such as classes, constructors, getters, and object interaction
- Use 2D arrays for efficient seat management
- Implement booking validation to prevent double booking
- Calculate total ticket cost based on seat categories
- Display a clear booking confirmation with movie name, seats, and cost

Scope

This project is suitable for academic demonstrations of Java OOP and can serve as a foundation for a fully-fledged cinema booking application. The current scope covers a single-screen, single-session booking flow with console-based interaction. Future extensions could include multi-screen support, database-backed persistence, a GUI interface, and user

authentication.

System Requirements

Hardware:

- Computer with minimum 4 GB RAM
- Any modern processor (Intel i3 / AMD equivalent or above)

Software:

- Java JDK 8 or above
- IDE: VS Code / Eclipse / Edit Plus / IntelliJ IDEA
- Operating System: Windows / Linux / macOS

Methodology

The system follows a straightforward workflow. On launch, available movies and their show timings are displayed. The user selects a movie, after which the current seat layout is rendered as a 2D grid. The user then inputs the row and column of the desired seat. The system checks whether the seat is already booked; if it is, an error message is shown and the user is prompted again. If the seat is free, it is marked as booked, its price is added to the running total, and the user may select additional seats. Once selection is complete, a full booking confirmation is displayed including the movie name, all selected seat coordinates, and the total cost.

Flowchart / Class Diagram

The booking flow starts at the main menu, proceeds through movie selection, seat layout display, seat selection with validation, cost calculation, and ends with booking confirmation.

START
Display Available Movies
User Selects a Movie
Display Seat Layout (2D Grid)
User Enters Row & Column
Is Seat Already Booked?
YES → Show Error & Re-prompt | NO → Mark Seat as Booked
Add Seat Price to Total Cost
Book More Seats?
YES → Back to Seat Layout | NO → Display Booking Confirmation
END

Class Diagram:

Movie	Seat	Booking

Movie	Seat	Booking
- movieName: String - showTime: String + Movie(name, time) + getMovieName() + getShowTime()	- type: String - price: double - isBooked: boolean + Seat(type, price) + isBooked() + book() + getPrice()	- seats[][]: Seat - totalCost: double + Booking(rows, cols) + bookSeat(row, col) + displayLayout() + displayConfirmation()

Implementation

The project is built around three Java classes, each with a clearly defined responsibility:

1. Movie Class

Stores the movie name and show time. It has a constructor to initialise these fields and getter methods (*getMovieName()*, *getShowTime()*) to expose the data to other classes.

2. Seat Class

Represents a single seat in the cinema. Each seat has a type (Regular or Premium), a price, and a boolean flag *isBooked*. The *book()* method sets *isBooked* to true once a seat is reserved. Regular seats are initialised with a lower price and Premium seats with a higher price.

3. Booking Class

Manages the 2D array of Seat objects (*Seat[][] seats*). The *bookSeat(int row, int col)* method first checks if the seat is already booked. If it is, it displays an error message. If not, it calls *seat.book()* and adds the seat price to *totalCost*. The *displayLayout()* method prints 'O' for available and 'X' for booked seats. Finally, *displayConfirmation()* prints the movie name, selected seats, and total cost.

Sample Input / Output

Below is an example of the expected terminal output of the system:

```
Available Movies:
1. Avengers: Endgame | Show Time: 6:00 PM

Seat Layout:
O O O O
O O O O
O O O O

Enter row and column to book (e.g. 1 2): 1 2
Seat (1,2) booked successfully!

Enter row and column to book (e.g. 1 2): 2 4
Seat (2,4) booked successfully!

Updated Seat Layout:
O X O O
O O O X
O O O O

---- Booking Confirmation ----
```

```
Movie : Avengers: Endgame
Show : 6:00 PM
Seats : (1,2), (2,4)
Total : Rs.500
Booking Confirmed!
```

Future Enhancements

- Support for multiple movies and show timings in a single session
- Row-based dynamic pricing (e.g. front rows cheaper, back rows premium)
- Cancel booking feature to free up reserved seats
- User input via Scanner class for fully interactive console experience
- GUI-based seat layout using Java Swing or JavaFX (advanced)
- Database integration (MySQL / SQLite) for persistent booking records
- User authentication with login and booking history

Gantt Chart

The table below shows the planned timeline for the Movie Ticket Booking System project, divided into key phases across a four-week period.

Task	Week 1	Week 2	Week 3	Week 4
Project Planning & Requirements	■■■■■			
Class Design (Movie, Seat, Booking)	■■■	■■		
Core Coding (Classes & Methods)		■■■■■	■■	
Booking Logic & Validation		■■	■■■	
Testing & Debugging			■■■	■■
Report Writing & Formatting			■■	■■■
Final Submission				■■■■■

Conclusion

The Movie Ticket Booking System successfully demonstrates the practical application of Object-Oriented Programming concepts in Java. By modelling real-world entities as classes — Movie, Seat, and Booking — the system achieves a clean, modular design that mirrors professional software architecture.

The use of 2D arrays for seat management, combined with robust booking validation, ensures data integrity and a reliable user experience. This project lays a strong foundation for more advanced features such as a GUI, database persistence, and multi-session support, making it a scalable starting point for a production-ready cinema booking application.

References

[1] H. Schildt, *Java: The Complete Reference*, 11th ed. New York: McGraw-Hill, 2018.

2 E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Boston: Addison-Wesley, 1994.

3 Oracle, "Java Documentation," Oracle. [Online]. Available: https://docs.oracle.com

4 GeeksforGeeks, "Java 2D Array," GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/multidimensional-arrays-in-java/

5 S. Kumar, "OOP Concepts in Java," in Proc. IEEE Conf. on Software Engineering Education, 2021, pp. 88-93.